

Curso Inicial Front-End

EL ECOSISTEMA WEB Y FUNDAMENTOS DE HTML

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)

Clase 1: Introducción al desarrollo web + Introducción a HTML (sintaxis)



Presentación

¡Bienvenidos al punto de partida de tu carrera como desarrollador! En esta clase aprenderemos **HTML**, el lenguaje que estructura la información de toda la web. Configuraremos tu entorno de trabajo profesional y entenderás por qué un código bien estructurado es la base de cualquier sitio exitoso.

Si estás leyendo esto, acabas de dar el primer paso para dejar de ser un simple consumidor de internet y convertirte en un creador. Hoy no solo escribiremos código, sino que entenderemos **qué es la web** y cómo se construye desde sus cimientos. No daremos nada por sentado.



Objetivos

- Configurar VS Code con herramientas de productividad industrial (AutoSave y Extensiones).
- Dominar la sintaxis de etiquetas, atributos y el anidamiento correcto.
- Diferenciar físicamente los elementos de bloque y de línea para maquetar con precisión.



Bloques Temáticos

- Tema 1: Arquitectura cliente/servidor y protocolos web.
 - El Ecosistema Web
 - Configurando tu Entorno de Trabajo
- Tema 2: Funcionamiento web y alcance de la diplomatura.
 - ¿Qué es HTML?
 - Puesta en Marcha: Tu Primer Archivo
 - La Estructura Obligatoria (El Boilerplate)
 - Anatomía de una Etiqueta (Tag)
 - El Comportamiento: Bloque vs. En Línea
 - Etiquetas básicas de Texto y Semántica

Tema 1: Arquitectura cliente/servidor y protocolos web.

1.1. El Ecosistema Web

Antes de levantar los cimientos de una casa o tirar la primera línea de código, es fundamental conocer el terreno sobre el que vamos a construir. En el desarrollo Front-End, ese terreno es internet. Para entender el software actual, primero debemos comprender cómo evolucionó el entorno web y qué procesos invisibles se desencadenan cada vez que interactuamos con el navegador.

- **La Evolución:**

La Web no siempre funcionó como la conocemos hoy. Su historia se divide en tres grandes eras tecnológicas:

- **Web 1.0 (90s):** Eran páginas estáticas, de solo lectura, como enciclopedias aburridas. (Puro **HTML**).
 - **Web 2.0 (2000s):** Nace la interactividad, las redes sociales y el contenido creado por usuarios. (**JavaScript** toma el control).
 - **Web 3.0 (Hoy):** La era de la descentralización, la web semántica y la Inteligencia Artificial.
- **¿Cómo funciona Internet? (HTTP)**

Cada acción que realizamos en internet se rige por la arquitectura Cliente/Servidor mediante el protocolo HTTP. Cuando un usuario escribe una URL (por ejemplo, google.com, lo que técnicamente llamamos Dominio), el navegador web actúa como el Cliente y envía una petición (Request) a través de la red.

Esta petición viaja hasta una computadora de alta capacidad encendida de forma ininterrumpida (24/7), conocida como Servidor o Hosting. El servidor procesa la solicitud y devuelve una respuesta (Response) que incluye el paquete de código fuente (HTML, CSS y JS) necesario para que el navegador dibuje la pantalla.

Junto con este paquete de datos, el servidor envía un Código de Estado HTTP, que indica el resultado de la operación:

- **200**: Todo OK (Carga exitosa).
- **404**: Página no encontrada (Te equivocaste en la URL).
- **500**: El servidor explotó (Error interno de la computadora lejana).

| Librería vs Framework (Concepto Vital)

A medida que las aplicaciones web de la era 2.0 y 3.0 se volvieron complejas, la industria creó herramientas para optimizar el desarrollo. Es crucial distinguir entre dos conceptos vitales en el mercado laboral:

Característica	Librería (ej: <u>React</u>)	Framework (ej: <u>Angular</u>)
Control	Tú llamas a la librería cuando la necesitas. Tú tienes el control.	El framework te llama a ti. Él dicta la arquitectura.
Tamaño	Pequeña, resuelve un problema específico (ej: Interfaces).	Enorme, incluye herramientas para todo (Ruteo, HTTP, Seguridad).
Curva de Aprendizaje	Más rápida y flexible.	Más lenta y estricta.

Comprender este flujo de peticiones, respuestas y herramientas es el primer paso para transformar simples usuarios de internet en profesionales del desarrollo web. Cada línea de código que escribamos estará destinada a viajar por esta infraestructura. Una vez que entendemos cómo opera este ecosistema, el siguiente paso lógico es preparar nuestra propia computadora para convertirnos en creadores de tecnología.

1.2. Configurando tu Entorno de Trabajo

Para materializar el software y comunicarnos de forma eficiente con la computadora, un desarrollador profesional no utiliza procesadores de texto comunes. Necesitamos configurar un ecosistema de herramientas estándar que optimicen la escritura de código, automaticen tareas repetitivas y simulen el entorno de un servidor real.

Paso 1: Instalación del Software Base.

Para escribir código y comunicarte con la computadora, necesitas instalar 3 herramientas fundamentales. Por favor descárgalas e instálalas usando las opciones por defecto ("Siguiente, Siguiente..."):

1. **Visual Studio Code (El Editor):** Es el programa donde escribiremos nuestro código. Es gratuito, creado por Microsoft y el estándar absoluto en la industria. > 📦 **Descarga:** code.visualstudio.com
2. **Git Bash (La Terminal):** Windows trae una terminal por defecto (CMD/PowerShell), pero los desarrolladores usamos comandos de Linux. **Git** Bash nos permite hablarle a la computadora en el idioma universal de los programadores y nos habilita para usar **GitHub** más adelante. > 📦 **Descarga:** git-scm.com/downloads (Descarga **Git** para tu sistema operativo).
3. **Node.js (El Motor):** Aunque empezaremos usando el navegador, más adelante (y en el backend) necesitaremos que nuestra computadora entienda **JavaScript** de forma nativa. Su instalación incluye **npm**, la tienda de herramientas del desarrollador. Descarga siempre la versión "LTS" (Recomendada). > 📦 **Descarga:** nodejs.org

Paso 2: Activación del Guardado Automático.

Con el editor ya instalado, el siguiente paso es adaptarlo a un ritmo de producción profesional. Para que tus cambios se vean en tiempo real sin tener que presionar **Ctrl + S** constantemente, activa el **AutoGuardado**: 1. Ve a la pestaña superior **File** (Archivo). 2. Selecciona la opción **Auto Save** (Autoguardado). Ahora, cada letra que escribas se guardará automáticamente, permitiendo que herramientas como Live Server actualicen la web al instante.

Paso 3: Equipamiento del Kit de Extensiones. El último paso consiste en potenciar nuestro editor transformándolo en un asistente inteligente. Instala estas extensiones desde el menú lateral de VS Code (icono de cubos) para trabajar como un profesional:

1. **Prettier - Code formatter:** Ordena tu código automáticamente para que sea legible.
2. **Auto Rename Tag:** Cambia la etiqueta de cierre automáticamente cuando editas la de apertura.
3. **ES7+ React/Redux/React-Native snippets:** Atajos de teclado para escribir código de React velozmente.
4. **ESLint:** Te avisa si estás cometiendo errores de sintaxis o lógica.
5. **Live Server:** Abre un servidor local para ver tus cambios en el navegador en tiempo real.
6. **Live Share:** Permite que otros programadores entren a tu código para ayudarte (ideal para tutorías).

Seguir estos tres pasos marca la diferencia entre un enfoque amateur y el perfil de un desarrollador profesional. Al automatizar el guardado, delegar el formato visual en las extensiones y contar con una consola estandarizada, eliminamos las distracciones operativas. Con la infraestructura de desarrollo correctamente desplegada en la computadora, estamos listos para dar el salto definitivo al

Tema 2: crear y renderizar nuestro primer archivo web de la diplomatura.

Tema 2: Funcionamiento web y alcance de la diplomatura.

2.1. ¿Qué es HTML?

Ahora que tenemos nuestro entorno de trabajo configurado, es momento de adentrarnos en el código. Sin embargo, antes de empezar a tipear sin sentido, debemos entender qué estamos escribiendo. Para construir interfaces web profesionales, el primer paso es comprender la naturaleza del lenguaje que le da vida a la estructura de internet y cómo se complementa con las demás tecnologías del mercado.

HTML significa **HyperText Markup Language** (Lenguaje de Marcado de Hipertexto). No es un lenguaje de programación matemático; no puede sumar **2+2**. Es un lenguaje de "etiquetas".

- *HyperText (Hipertexto)*: Es texto que contiene enlaces a otros textos. Forma una red conectada.
- *Markup (Marcado)*: Usamos "marcas" o etiquetas para decirle al navegador qué rol cumple cada cosa ("Oye, esto es un título, y esto es una foto").

El Navegador (Cliente), a través de programas como Chrome o Firefox, actúa como un intérprete. Ellos leen tu código oculto y lo "traducen" en la pantalla visual que ves. Para profundizar en sus bases, siempre podés consultar el

Recurso Oficial: [MDN Web Docs - HTML Basics](https://developer.mozilla.org/es/docs/Web/HTML)

Para entender su alcance en el desarrollo de software, analicemos sus ventajas y limitaciones:

✓ Ventajas (Pros)	✗ Desventajas (Contras)
Universal: Funciona nativamente en cualquier navegador del mundo sin instalar nada.	Estático: No puede calcular, sumar, ni tomar decisiones lógicas por sí solo.
Accesible: Es la base fundamental para el SEO (Google) y lectores de pantalla.	Básico Visualmente: Sin CSS adicional, se ve feo, tosco y en blanco y negro.
Fácil de aprender: Su sintaxis es predecible y muy intuitiva.	No mantiene el estado: No puede "recordar" datos del usuario por su cuenta.

Los 3 Pilares del Frontend: El Concepto "LEGO"

Imagina que construyes una casa de LEGO. Para que sea habitable y linda, necesitas 3 herramientas inseparables:

Herramienta	Analogía	Función Real
<u>HTML</u>	Los Bloques básicos	Define el CONTENIDO (¿Qué hay aquí? ¿Es un texto o un botón?).
<u>CSS</u>	Las Pinturas y decoraciones	Define la ESTÉTICA (¿De qué color es? ¿Dónde está ubicado?).
<u>JS</u>	Los Motores y luces	Define la INTERACCIÓN (¿Qué pasa si hago clic? ¿Se mueve?).

En lenguaje simple: **HTML** es el esqueleto de la página, **CSS** es la ropa y decoración, y **JS** es el cerebro que le dice qué hacer cuando alguien interactúa con ella.

Traducción Técnica: Esta tríada constituye la arquitectura de un documento web moderno. El **HTML** (DOM) define los nodos y la jerarquía de objetos. El **CSS** (CSSOM) aplica reglas de cascada y especificidad para calcular el renderizado visual. El **JavaScript** (Engine) manipula programáticamente ambos modelos de objetos en tiempo de ejecución para crear dinamismo.

Como pudimos ver, HTML no trabaja solo; es la pieza fundacional sobre la que se apoya toda la experiencia de usuario en el Front-End. Dominar este esqueleto es lo que te va a permitir, más adelante, aplicar estilos complejos y lógicas interactivas avanzadas. Ahora que entendemos conceptualmente cómo funciona esta estructura, pasemos de la teoría a la práctica para crear nuestro primer documento web real.

2.2. Puesta en Marcha: Tu Primer Archivo

Ya tenemos las herramientas instaladas y conocemos los fundamentos teóricos del lenguaje, ¡así que manos a la obra! Para crear una página web no necesitas magia ni configuraciones complejas, solo un archivo de texto especial. Para dar este primer paso sin frustrarte en el intento, sigue estos pasos exactos en tu computadora:

1. **Crea tu mesa de trabajo:** En el escritorio de tu computadora, haz clic derecho y crea una nueva carpeta llamada `mi-primer-web`.
2. **Abre tu editor:** Abre el programa *Visual Studio Code* y arrastra esa carpeta recién creada hacia el centro de la pantalla negra del programa.
3. **Crea el archivo mágico:** En el panel izquierdo de VS Code, haz clic en el ícono de "Nuevo Archivo" y nómbralo exactamente `index.html`. (Nota de color: "index" es el nombre universal que buscan los servidores para saber cuál es la página principal, nunca le pongas "inicio.**html**").

4. **El atajo secreto:** Escribe el signo de exclamación `!` y presiona la tecla `Tab` o `Enter`. VS Code escribirá mágicamente toda la estructura base obligatoria por ti. (Veremos qué es esta estructura más abajo).
5. **Visualízalo (Renderizar):** Para ver tu código dibujado en la pantalla (a este proceso de dibujar el código lo llamamos **Renderizar**), busca tu archivo `index.html` en la carpeta de tu escritorio y dale doble clic. Se abrirá en tu navegador (Chrome/Edge) como una pantalla en blanco, lista para recibir código.

¡Felicidades! Acabas de maquetar y poner en marcha tu primer archivo en un entorno de desarrollo real. Aunque el navegador todavía se vea en blanco porque no le hemos agregado contenido visible, este archivo ya cuenta con la estructura técnica necesaria para ser interpretado en cualquier parte del mundo. En el siguiente apartado, abriremos este "archivo mágico" para analizar con lupa qué significa cada una de las líneas de código que VS Code generó automáticamente.

2.3. La Estructura Obligatoria (El Boilerplate)

¿Recuerdas el atajo `!` que usamos en el paso 3 de la Puesta en Marcha? Eso generó un montón de código en inglés. A ese código base se le llama "Boilerplate". Todo archivo **HTML**, por más simple que sea, lo necesita. Si falta, los motores de búsqueda te ignorarán.

Para entender qué está pasando por detrás de la pantalla, vamos a desglosar y traducir pieza por pieza qué significa cada una de estas líneas fundamentales:

1. `<!DOCTYPE html>` **(Declaración de tipo):** No es una etiqueta, es un grito al navegador: "*¡Atención, este documento está escrito en la última versión, HTML5!*".

2. `<html lang="es">` **(La raíz):** Es el contenedor madre. El atributo `lang="es"` es crítico para que los lectores de pantalla (no videntes) y los traductores automáticos sepan que tu página habla en español.
3. `<head>` **(La Cabeza - Datos ocultos):** Contiene la "meta-información". **Nada** de lo que pongas aquí se verá en la página. Es información técnica exclusiva para Facebook, Google y el navegador.
 - `<meta charset="UTF-8">`: Es el estándar que permite que tu web entienda tildes (á, é) y emojis. Sin esto, verías símbolos rotos ``.
 - `<meta name="viewport" ...>`: Es el "Botón Mágico" del diseño adaptable. Le ordena a la página que su ancho se adapte automáticamente a la pantalla del celular.
 - `<title>`: El texto superior que aparece en la pestaña del navegador.
4. `<body>` **(El Cuerpo visible):** Aquí es donde entramos nosotros. Todo el texto, imágenes, botones y videos que el usuario final verá debe ir estrictamente dentro de esta etiqueta.

El Código Limpio (Cómo se ve tu `index.html` ahora mismo):

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Primera Web</title>
</head>
<body>
  <!-- CONTENIDO VISIBLE: Todo lo que pongas aquí aparecerá en la
pantalla blanca -->
</body>
</html>
```

El *Boilerplate* representa el contrato técnico que firmamos con el navegador para garantizar que nuestro software sea accesible, moderno y adaptable. Una vez montada esta estructura obligatoria, dejamos de lado las configuraciones

técnicas invisibles del `<head>` y nos enfocamos en el `<body>`, donde empezaremos a dar forma a la información utilizando las reglas de escritura del lenguaje.

2.4. Anatomía de una Etiqueta (Tag)

Casi todo en HTML funciona envolviendo texto o datos mediante etiquetas. Para maquetar interfaces con precisión profesional y evitar fallos visuales en el navegador, es indispensable comprender las reglas físicas que rigen a estas etiquetas, cómo se les inyecta información extra y de qué manera se estructuran los caminos para vincular archivos entre sí.

Una etiqueta completa (a la que en la jerga técnica llamamos un **Elemento**) se compone habitualmente de tres partes fundamentales que deben respetarse en orden:

1. **Etiqueta de apertura:** `<p>` (Abre la instrucción).
2. **Contenido:** El texto o dato real.
3. **Etiqueta de cierre:** `</p>` (Nota la barra diagonal `/`, que indica que la instrucción terminó).

Ejemplo: `<p>Hola Mundo</p>` ➡ El navegador traduce esto renderizando un párrafo.

Configuración y Enlaces: Las Reglas de los Elementos

Una vez que comprendemos la estructura básica de apertura y cierre, es necesario dominar tres componentes avanzados que transforman etiquetas simples en herramientas potentes y funcionales:

1. Atributos (Inyectando Información Extra): A veces, las etiquetas básicas se quedan cortas y necesitan instrucciones especiales para poder funcionar. Se llaman **Atributos** y se declaran siempre de forma exclusiva dentro de la etiqueta de apertura utilizando el formato `nombre="valor"`:

```
<a href="https://google.com" target="_blank">Ir a Google</a>
```

- `href`: Es el atributo de "destino" (¿A dónde voy?).
- `target="_blank"`: Es un atributo extra que le dice al navegador: "Abre este enlace en una pestaña nueva, no borres mi página".

2. Rutas Relativas vs. Absolutas (El Mapa del Navegador): Cuando usas atributos como `href` o `src` (para buscar imágenes), debes indicarle al navegador dónde buscar ese archivo:

- **Ruta Absoluta:** La dirección completa en internet (ej: `https://.../foto.jpg`). Siempre empieza con `http` y se usa para recursos externos.
- **Ruta Relativa:** El camino paso a paso desde el archivo donde estás parado hasta el destino.
- `./foto.jpg`: "Busca en esta misma carpeta donde estoy yo".
- `../foto.jpg`: "Sube una carpeta hacia atrás y busca ahí". ¡Esto es vital para organizar proyectos grandes!

3. Elementos Vacíos (Self-closing): Como curiosidad, no todas las etiquetas necesitan texto adentro. Algunas son "autónomas" y no tienen etiqueta de cierre:

- ``: Muestra imágenes.
- `<br>`: Fuerza un salto de línea (como presionar Enter en Word).
- `<hr>`: Dibuja una línea horizontal divisoria.

Conocer la anatomía de las etiquetas y el manejo de rutas te da el control absoluto sobre cómo viaja la información dentro de tu aplicación web. Evitar errores comunes de sintaxis —como olvidar una barra de cierre o tipear mal una ruta relativa— es lo que distingue a un maquettador sólido en entornos de

trabajo reales. Con estas reglas claras, estamos listos para analizar cómo se comportan físicamente estos elementos al acomodarse en la pantalla del navegador.

2.5. Elementos de Bloque vs. En Línea

Al maquetar interfaces web profesionales, no basta con conocer la sintaxis de las etiquetas; debemos comprender cómo se comportan físicamente en la pantalla. En HTML, cada etiqueta tiene una "personalidad" o comportamiento físico predeterminado que define la forma en que se posiciona respecto a sus elementos vecinos. Dominar esta distinción evita que el diseño "se rompa" y nos permite estructurar layouts con absoluta precisión visual.

Para entender las reglas que gobiernan el espacio en el navegador, analicemos las diferencias clave entre ambos comportamientos:

Característica	Elementos de Bloque (<i>block</i>)	Elementos en Línea (<i>inline</i>)
Ancho	Ocupan el 100% del ancho disponible.	Ocupan solo el ancho de su contenido.
Salto de Línea	Siempre empiezan en una línea nueva.	Se colocan uno al lado del otro.
Ejemplos	<code><div></code> , <code><h1></code> , <code><p></code> , <code></code> , <code><footer></code>	<code></code> , <code><a></code> , <code></code> , <code></code>

Consejos e Interpretación Profesional



TIP

Analogía: Los elementos de **bloque** son como cajas de mudanza (necesitan su propio espacio). Los elementos **en línea** son como libros en una estantería (puedes poner varios uno junto al otro).

En lenguaje simple: Los elementos de bloque son como ladrillos que ocupan todo el ancho y se apilan, mientras que los de línea son como palabras que se acomodan una al lado de la otra.

Traducción Técnica: El comportamiento de **bloque** implica que el elemento genera una caja que se expande horizontalmente para ocupar todo el contenedor padre (width: 100%) y fuerza un salto de línea en el flujo normal del documento. El comportamiento **en línea** solo ocupa el ancho intrínseco de su contenido y no permite modificar propiedades de dimensiones verticales como `height` o `margin-top/bottom` de la misma manera que los bloques.

⚠ **IMPORTANT**

Regla de Oro Arquitectónica: Puedes meter un elemento "En Línea" dentro de uno de "Bloque" (ej: un link chiquito adentro de un gran párrafo), pero **NUNCA** debes meter un elemento gigante de "Bloque" (como un `<div>` o un `<h1>`) dentro de uno pequeño "En Línea" (como un `<a>` o un `<span>`).

Entender cómo interactúan las cajas de bloque y los elementos en línea constituye la base física de la maquetación. Con estas reglas de convivencia claras, el siguiente paso es conocer el catálogo de etiquetas fundamentales que utilizaremos en el día a día para organizar el texto y dotar a la página de un sentido lógico e inteligente.

2.6. Etiquetas básicas de Texto y Semántica

Ya sabemos dónde escribir (adentro del `<body>`). Ahora, repasaremos las etiquetas fundamentales para darle forma a nuestro texto. Escribe el **HTML** para ayudar al robot de Google a entender qué es importante, no para hacer que las cosas se vean "bonitas" (para lo bonito usaremos **CSS** luego).

Para estructurar un documento accesible y optimizado para SEO (posicionamiento en buscadores), disponemos del siguiente set de etiquetas semánticas:

- **Encabezados (h1 al h6):** Para títulos. El **h1** es el más importante (el título del diario) y solo debe haber UNO por página. Ayuda muchísimo al SEO (Posicionamiento en buscadores).
- **Párrafos (p):** Para bloques de texto normal largo.
- **Enlaces (a):** Significa "Anchor" (Ancla). Sirve para crear links. Recuerda siempre usar su atributo vital **href**.
- **Texto Enfatzado y Negritas:**
 - **** vs ****: Ambos se ven "en negrita" visualmente, pero **** le dice a Google que esa palabra es **crítica semánticamente**, mientras que **** es solo decoración sin valor. ¡Usa siempre ****!
 - **** vs **<i>**: Ambos se ven "en cursiva", pero **** indica énfasis hablado (como cuando levantas la voz), y la **<i>** es solo estilo visual.
- **Listas (ul, ol y li):**
 - **** (Unordered List): Crea una lista desordenada (con viñetas o puntitos).
 - **** (Ordered List): Crea una lista ordenada (numerada 1, 2, 3...).
 - **** (List Item): Representa a CADA elemento de la lista. **Obligatoriamente** deben ir envueltos dentro de un **ul** o un **ol**.

Ejemplo Práctico en Código (¡Cópialo en tu body y pruébalo!):

Copía el siguiente bloque directamente dentro del **<body>** de tu archivo **index.html** en VS Code y visualizá cómo el navegador interpreta las jerarquías y los comportamientos en línea y en bloque:

```
<h1>Biografía de Marie Curie</h1>
<p>Marie Curie fue una física y química polaca, pionera en el campo de la
<strong>radiactividad</strong>.</p>
<p>Si quieres saber más, visita su <a
href="https://es.wikipedia.org/wiki/Marie_Curie" target="_blank">página de
Wikipedia</a>.</p>

<h3>Sus mayores logros:</h3>
<ul>
  <li>Primera persona en recibir dos premios Nobel.</li>
  <li>Descubrió el <em>Polonio</em> y el <em>Radio</em>.</li>
</ul>
```

El uso correcto de estas etiquetas básicas marca la diferencia entre un programador que solo amontona texto y un desarrollador profesional que construye código semántico, accesible y preparado para escalar. Al dominar la combinación de títulos, párrafos, listas y enlaces estructurados bajo las reglas de bloques y líneas, has adquirido las competencias esenciales del maquetado Front-End. Ahora es tu turno de poner a prueba estos conocimientos en el entorno real de desarrollo.

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **Git:** <https://git-scm.com/doc>
- **GitHub:** <https://docs.github.com/es>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **Node.js:** <https://nodejs.org/es/docs/>
- **React:** <https://es.react.dev/>